

Academy Playlists API

Backend Assignment — Technical Documentation

ASP.NET Core 10 • Entity Framework Core • SQLite • Swagger

github.com/welo154/AcademyBackendTest

1. Overview

The Academy Playlists API is a small backend service built for the Academy backend assignment. It's written in ASP.NET Core 10 with Entity Framework Core on top of SQLite, and it's fully documented through Swagger.

The core idea is simple: a user can create playlists, and each playlist can hold songs. Alongside the required create/read functionality, delete endpoints were added for playlists and songs as a bonus.

- Create a user
- Create a playlist for a user
- Add songs to a playlist
- Fetch all playlists belonging to a user
- Delete a playlist (and its songs, via cascade delete)
- Delete a single song from a playlist

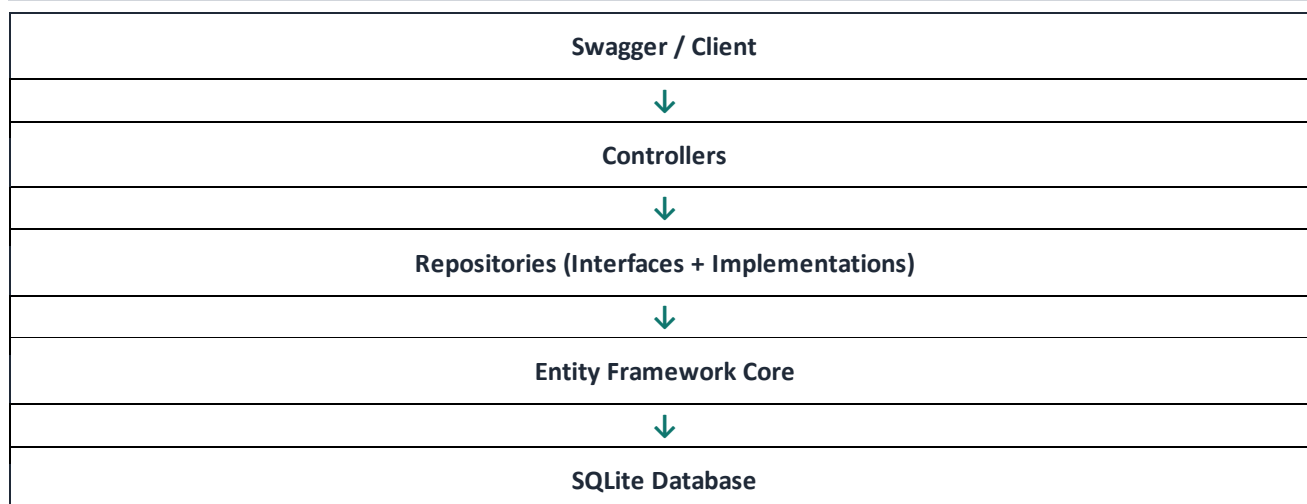
2. Tech Stack

- ASP.NET Core 10 (C#)
- Entity Framework Core — ORM and migrations
- SQLite — file-based relational database
- Swagger / OpenAPI — interactive API documentation
- Git & GitHub — version control

SQLite was chosen mainly for convenience: it needs no separate server, it's easy to clone and run anywhere, and it still models the relational shape of the data properly — one user has many playlists, and one playlist has many songs. For a real production system, something like PostgreSQL or SQL Server would replace it.

3. Architecture

The project follows a straightforward layered architecture, keeping HTTP concerns, business logic and data access separate:



Patterns used

- Repository Pattern — data access is wrapped behind IUserRepository / IPlaylistRepository interfaces, so controllers never talk to Entity Framework directly.
- Dependency Injection — repositories are registered in Program.cs and injected wherever they're needed.
- DTOs — requests and responses use dedicated DTO classes instead of exposing the EF models directly.
- Controllers — thin, and responsible only for routing, validation, and returning the right status codes.

SOLID principles

The layered design was also a chance to apply SOLID principles rather than just talk about them:

Principle	How it shows up	Why it matters here
Single Responsibility	Controllers only handle HTTP; repositories only handle data access	Each class has one reason to change
Open/Closed	New repositories can be added without touching existing controllers	New features extend the app rather than rewrite it
Liskov Substitution	Any class implementing IUserRepository / IPlaylistRepository can replace another	Implementations are interchangeable behind the interface
Interface Segregation	Separate, focused interfaces per entity instead of one large repository	Controllers only depend on the methods they actually use
Dependency Inversion	Controllers depend on repository interfaces, injected via the built-in DI container	Business logic isn't tied to Entity Framework directly

4. Database Schema

Three tables, two relationships: a user owns many playlists, and a playlist owns many songs. Cascade delete is enabled, so removing a playlist removes its songs automatically.

Users

Column	Description
Id	Primary key
Name	User's display name

Playlists

Column	Description
Id	Primary key
Name	Playlist name
CreatedAt	Creation timestamp
UserId	Foreign key to Users

Songs

Column	Description
Id	Primary key
Title	Song title
Artist	Artist name
DurationInSeconds	Optional duration
PlaylistId	Foreign key to Playlists

5. API Endpoints

All routes are prefixed with `/api` and documented interactively through Swagger. A playlist must reference an existing user, and a song must reference an existing playlist — the API validates that before saving anything.

Method	Endpoint	Description
POST	<code>/api/users</code>	Create a user
POST	<code>/api/playlists</code>	Create a playlist
POST	<code>/api/playlists/{playlistId}/songs</code>	Add a song to a playlist
GET	<code>/api/playlists/user/{userId}</code>	Get all playlists for a user
DELETE	<code>/api/playlists/{playlistId}</code>	Delete a playlist
DELETE	<code>/api/playlists/{playlistId}/songs/{songId}</code>	Delete a song

Typical request flow



Swagger UI

Endpoints grouped by resource in the Swagger UI:



Figure 1 — Playlists and Users endpoint groups in Swagger.

Example of a request/response schema shown by Swagger, for the "create user" endpoint:

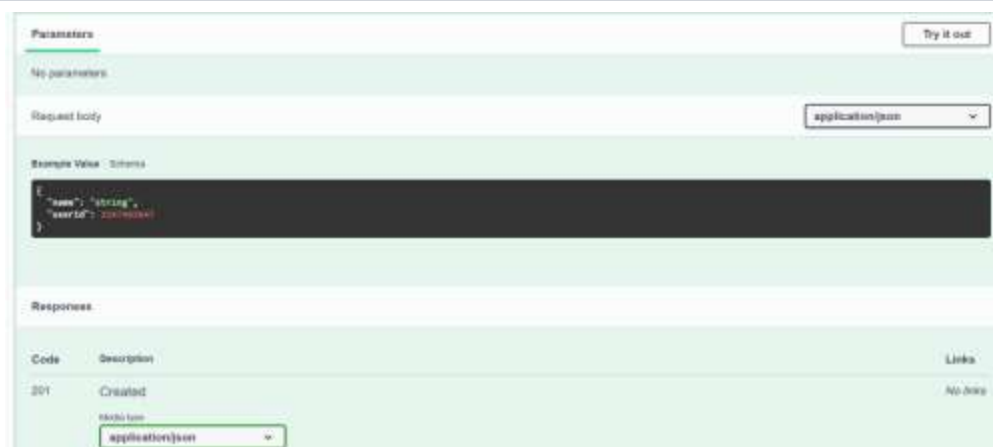


Figure 2 — Example request body and response codes for POST /api/users.

Example — creating a user

```
POST /api/users {  "name": "Ahmed" }
```

The response returns the new user's id, which is then used when creating a playlist for that user.

6. HTTP Status Codes

Status	Meaning
200	Request completed successfully
201	Resource created successfully
204	Resource deleted successfully
400	Invalid request
404	User, playlist, or song not found
500	Unexpected server error

7. Running the Project

Quick summary — full setup only needs the .NET 10 SDK and the EF Core CLI tool.

```
git clone https://github.com/welo154/AcademyBackendTest cd
AcademyBackendTest/Academy.Playlists.Api dotnet ef database update dotnet run
```

Once it's running, Swagger is available at the /swagger route on whatever port the terminal prints (usually something like localhost:5171).

8. AI Usage

AI tools were used during development mainly as a learning and debugging aid — understanding the assignment, scaffolding ASP.NET Core and EF Core, working through the Repository pattern and DI setup, and troubleshooting build/runtime errors. Every suggestion was reviewed and verified through Swagger before being kept. Also the AI helped me in understanding some .NET syntax as it is new language for me & also in implementing the Repository pattern in a clean way & in organizing and making this document. (All the AI chats context in AI USAGE.pdf file)